

Lab6 Control/Data Flow Analysis

1. controlflowinfo.cc

把Quad程序划分为基本块，形成 control flow graph，并计算它们的前驱、后继等得到支配信息。

1.1 computeAllBlocks

划分块，用 `set<int>allBlocks` 存储块编号的集合，`map<int,QuadBlock*> labelToBlock` 存编号到块的映射。扫描每个基本块，用 `entry_label` 作为块编号。

1.2 computeUnreachableBlocks

从 `entryBlock` 开始，进行 bfs，取队首的块出来，枚举所有的 `exit_labels`，如果没有在 `set<int>reachable` 的集合中，则加入集合，并把块编号入队。

枚举所有块，没有在集合中的块则加入 `unreachableBlocks` 的集合。

1.3 eliminateUnreachableBlocks

重新构建一个块列表 `vector<QuadBlock*>* newBlocks`，只保留可达块。然后更新 `func->quadblocklist`。清除其他前驱、后继等信息，重新调用 `computeAllBlocks` 得到干净的 CFG。

1.4 computePredecessors

遍历每个块，把自己插入到后继的前驱集合中。计算得到各个块的前驱集合。

1.5 computeSuccessors

把每个块的 `exit_labels` 插入自己的后继集合中。注意，由于前面删除了不可达块，因此要检查每个块位于 `allBlocks` 集合中才插入，不然是不可达块，丢弃掉。

1.6 computeDominators

采用迭代的方式。初始入口块 `dominators[entry] = {entry}`，其他块 `dominators[n] = allBlocks`。随后进行迭代：

对每个非入口块，把自己的 `dominators` 改为其所有前驱的 `dominators` 的交集并上它自己。循环迭代直到不再变化，得到每个块被哪些块支配。

1.7 computeImmediateDominator

计算每个块的 `immediateDominator`，即在所有支配B的块中，离B最近的块（除了B自己）。入口块的 `idom` 设为 -1。对于其他块，枚举它的 `strictDom`（排除掉自己的支配集合）。如果一个块不是集合中任何一个块的支配块，那么这个块就是最后一个，即最近的。

1.8 computeDomTree

构造支配树，把每个节点连到它的 `immediateDominator` 下面。

1.9 computeDominanceFrontiers

计算 Dominance Frontier，对于节点 x ，如果它支配了 w 的一个前驱，但是它不支配 w ，则 w 属于 $DF[x]$ 。

算法：考虑前驱 ≥ 2 的块 b ，枚举它的每个前驱 p ，只要 `idom[b] != p`，则把 b 加入 $DF[p]$ ，随后 `p=idom[p]`，沿着 `idom` 链向上移动。

2. dataflowinfo.cc

2.1 findAllVars

`set<int> allVars` 存储函数内所有变量的集合。`map<int, set<pair<quad::QuadBlock*, quad::QuadStm*>>> *defs/uses;` 存每个变量在哪些 (Block, Stm) 中被 def/use。

2.2 computeLiveness

计算每条语句的 live-in 和 live-out 集合。构建 label 到 block 的映射，方便从块末尾跳到后继块首语句。

然后进行循环迭代。初始所有 in/out 集合为空。每轮中 Block 从后往前，块中语句也从后往前迭代：

$$in[n] = use[n] \cup (out[n] - def[n]), \quad out[n] = \bigcup_{s \in succ[n]} in[s]$$

其中 `succ[n]`：若 n 不是块内最后一条，后继是下一条语句；若 n 是块内最后一条，后继是各 exit labels 对应后继块的首语句。

不变后得到各个点的 livein 和 liveout。

`livein[stm]`：执行 stm 前必须还活着的变量。

`liveout[stm]`：执行 stm 后仍可能被后续使用的变量。

2.3 dataFlowProg

把整个程序里的每个函数都做一遍数据流分析，并返回一个 `set<DataFlowInfo*>`。

对每个有效函数：

用 `placement new` 在内存池当前槽位构造 `DataFlowInfo(func)`，调 `findAllVars()` 收集变量定义/使用信息，调 `computeLiveness()` 计算 live-in/live-out。最后把 `DataFlowInfo` 插入 `allDataFlows`。遍历完所有函数后返回数据流信息的集合。

3. git graph

